

Home Made Pickles & Snacks: Taste the Best

Project Description:

Home Made Pickles & Snacks — Taste the Best is a cloud-based culinary platform revolutionizing access to authentic, handcrafted pickles and snacks. Addressing the growing demand for preservative-free, traditional recipes, this initiative combines artisanal craftsmanship with cutting-edge technology to deliver farm-fresh flavors directly to consumers. Built on Flask for backend efficiency and hosted on AWS EC2 for scalable performance, the platform offers seamless browsing, ordering, and subscription management. DynamoDB ensures real-time inventory tracking and personalized user experiences, while fostering sustainability through partnerships with local farmers and eco-friendly packaging. From tangy regional pickles to wholesome snacks, every product celebrates heritage recipes, nutritional integrity, and convenience—proving that tradition and innovation can coexist deliciously. "Preserving Traditions, One Jar at a Time."

Scenarios:

Scenario 1: Scalable Order Management for High Demand

A cloud-based system ensures seamless order processing during peak user activity. For instance, during a promotional event, hundreds of users simultaneously access the platform to place orders. The backend efficiently processes requests, updates inventory in real-time, and manages user sessions. The cloud infrastructure handles traffic spikes without performance degradation, ensuring smooth transactions and minimizing wait times.

Scenario 2: Real-Time Inventory Tracking and Updates

When a customer places an order for a product, the system instantly updates stock levels and records transaction details. For example, a user purchases an item, triggering automatic inventory deduction and order confirmation. Staff members receive updated dashboards to monitor stock availability and fulfillment progress, ensuring timely restocking and minimizing overselling risks..

Scenario 3: Personalized User Experience and Recommendations

The platform leverages user behavior data to enhance engagement. A returning customer, for instance, views tailored recommendations based on past purchases and browsing history. The system dynamically adjusts suggestions in real-time, while maintaining fast response rates even during high traffic, creating a frictionless and intuitive shopping experience.

AWS Technical Architecture:



- AWS Account Setup:
<https://docs.aws.amazon.com/accounts/latest/reference/getting-started.html>
- AWS IAM (Identity and Access Management):
<https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>
-

- AWS EC2 (Elastic Compute Cloud):
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>
- AWS DynamoDB:
<https://docs.aws.amazon.com/amazondynamodb/Introduction.html>
- Git Documentation:
<https://git-scm.com/doc>
- VS Code Installation: (download the VS Code using the below link or you can get that in Microsoft store)
<https://code.visualstudio.com/download>

1.Project Workflow:

Milestone 1. Backend Development and Application Setup

- Develop the Backend Using Flask.
- Integrate AWS Services Using boto3.

Milestone 2. AWS Account Setup and Login

- Set up an AWS account if not already done.
- Log in to the AWS Management Console

Milestone 3. DynamoDB Database Creation and Setup

- Create a DynamoDB Table.
- Configure Attributes for User Data and Book Requests.

Milestone 4. SNS Notification Setup

- Create SNS topics for book request notifications.
- Subscribe users and library staff to SNS email notifications.

Milestone 5. IAM Role Setup

- Create IAM Role
- Attach Policies

Milestone 6. EC2 Instance Setup

- Launch an EC2 instance to host the Flask application.
- Configure security groups for HTTP, and SSH access.

Milestone 7. Deployment on EC2

- Upload Flask Files
- Run the Flask App

Milestone 8. Testing and Deployment

- Conduct functional testing to verify user signup, login, buy/sell stocks and notifications.

Milestone 1: Web Application Development and Setup

Backend Development and Application Setup focuses on establishing the core structure of the application. This includes configuring the backend framework, setting up routing, and integrating database connectivity. It lays the groundwork for handling user interactions, data management, and secure access.

Important Instructions:

- Start by creating the necessary HTML pages and Flask routes (app.py) to build the core functionality of your application.
- During the initial development phase, store and retrieve data using Python dictionaries or lists locally. This will allow you to design, test, and validate your application logic without external database dependencies.
- Ensure your app runs smoothly with local data structures before integrating any cloud services.

Post Troven Access Activation:

- Once Troven Labs access is provided (valid for 4 hours), you must immediately proceed with Milestone 1 of your Guided Project instructions.

- At this point, modify your app.py and replace local dictionary/list operations with AWS services (such as DynamoDB, RDS, or others as per project requirements).
- Using the temporary credentials provided by Troven Labs, securely connect your application to AWS resources.
- Since the AWS configuration is lightweight and already instructed in the milestones, you should be able to complete the cloud integration efficiently within the allotted time.

Milestone 2: AWS Account Setup

Important Notice: Use Troven Labs for AWS Access

Students are strictly advised not to create their own AWS accounts, as doing so may incur charges. Instead, we have set up a dedicated section called “Labs” on the Troven platform, which provides temporary and cost-free access to AWS services.

Once your website is locally deployed and fully functional, you must proceed with integrating AWS services only through the Troven Labs environment. This ensures secure, controlled access to AWS resources without any risk of personal billing.

All steps involving AWS (such as deploying to EC2, connecting to DynamoDB, or using SNS) must be carried out within the Troven Labs platform, as we've configured temporary credentials for each student.

Reminder: You must complete the Web Development task before gaining access to Troven. Once accessed, the AWS Console via Troven is available for only 3 hours—please plan your work accordingly.

Please follow the provided guidelines and access AWS exclusively through Troven to avoid unnecessary issues.

Please refer the below link -

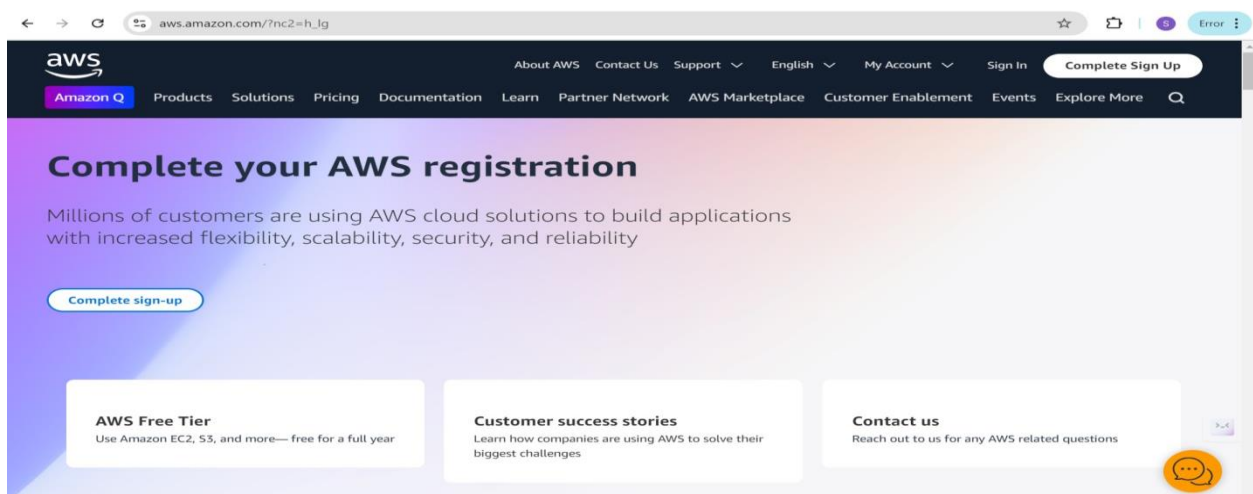
<https://drive.google.com/file/d/1HzWc7AMj2BrxhV-uaw5s0vWtcd-28qgl/view?usp=sharing>

AWS Account Setup and Login

This is for your understanding only, please refrain from creating an AWS account. A temporary account will be provided via Troven.

- Go to the AWS website (<https://aws.amazon.com/>).

- Click on the "Create an AWS Account" button.
 - Follow the prompts to enter your email address and choose a password.
 - Provide the required account information, including your name, address, and phone number.
 - Enter your payment information. (Note: While AWS offers a free tier, a credit card or debit card is required for verification.)
 - Complete the identity verification process.
 - Choose a support plan (the basic plan is free and sufficient for starting).
 - Once verified, you can sign in to your new AWS accounts.
-
- **Activity 2.1: Set up an AWS account if not already done.**
 - Sign up for an AWS account and configure billing settings.



- **Activity 2.2: Log in to the AWS Management Console**
 - After setting up your account, log in to the [AWS Management Console](#).



Sign in

☒ **Root user**

Account owner that performs tasks requiring unrestricted access. [Learn more](#)

☐ **IAM user**

User within an account that performs daily tasks. [Learn more](#)

Root user email address

Next

By continuing, you agree to the [AWS Customer Agreement](#) or other agreement for AWS services, and the [Privacy Notice](#). This site uses essential cookies. See our [Cookie Notice](#) for more information.

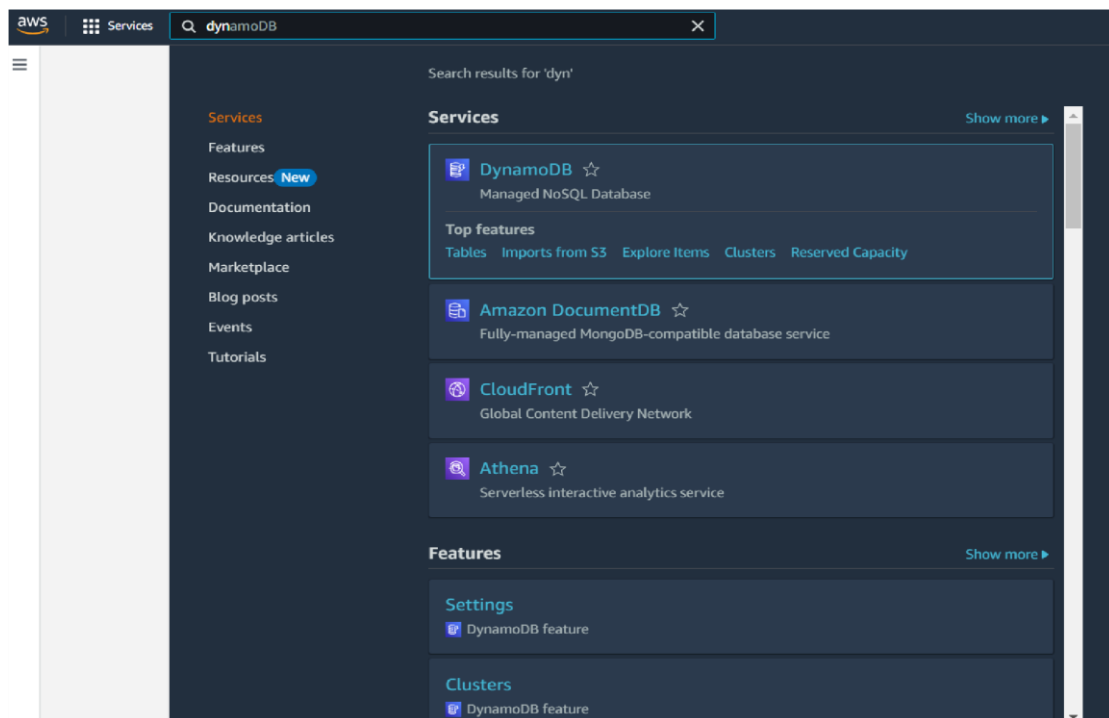
AI Use Case Explorer

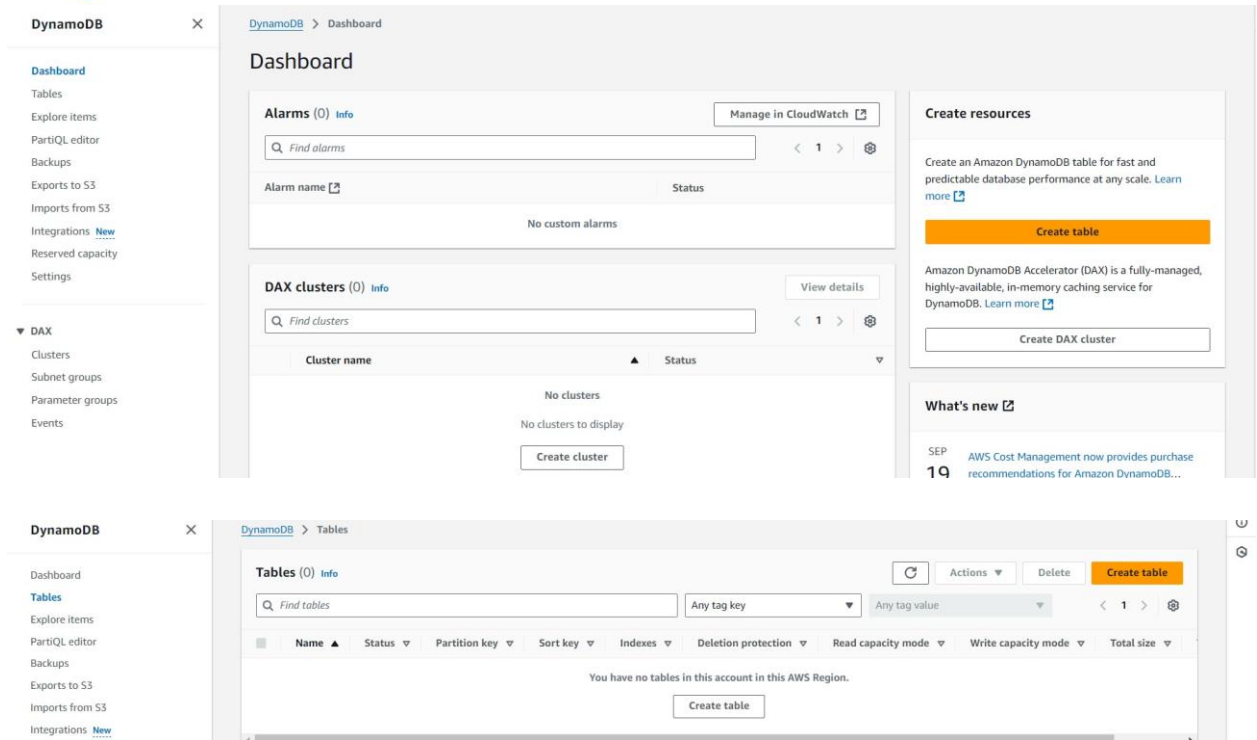
Discover AI use cases,
customer success stories,
and expert-curated
implementation plans

Explore now

Milestone 3: DynamoDB Database Creation and Setup

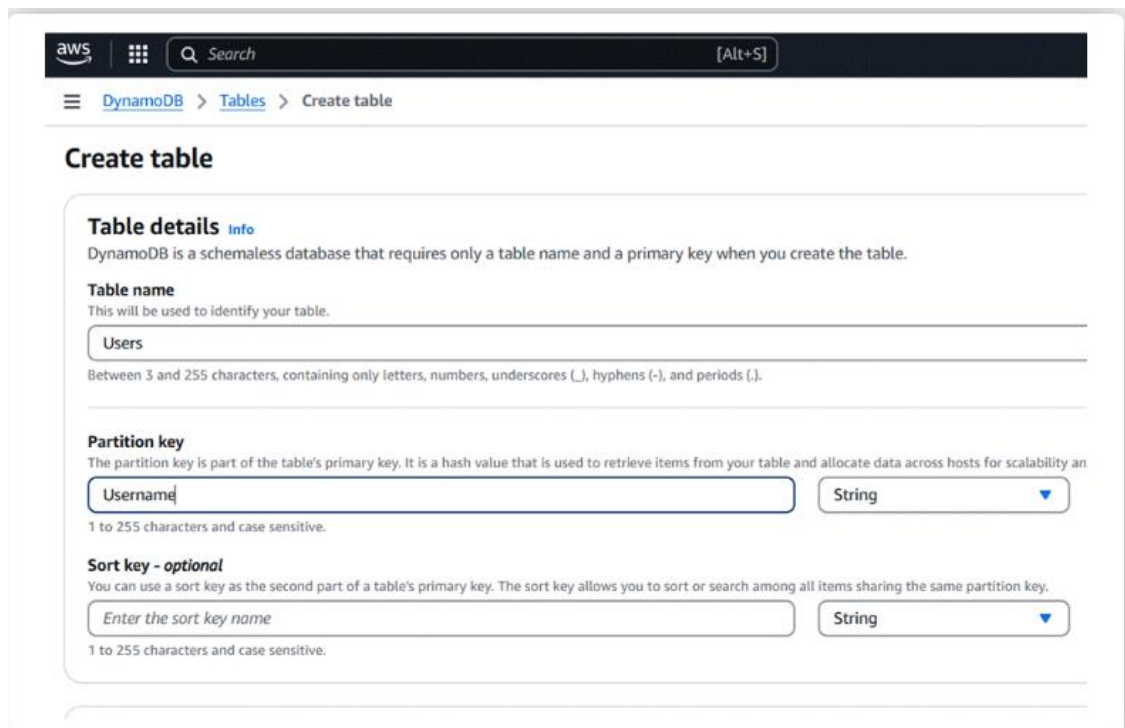
- **Activity 3.1: Navigate to the DynamoDB**
 - In the AWS Console, navigate to DynamoDB and click on create tables.





The screenshot shows the AWS DynamoDB console interface. The top navigation bar includes 'DynamoDB' and 'Dashboard'. The left sidebar lists various DynamoDB features like 'Tables', 'Explore items', ' PartiQL editor', 'Backups', 'Exports to S3', 'Imports from S3', 'Integrations', 'Reserved capacity', and 'Settings'. The main content area is divided into two sections: 'Alarms (0)' and 'DAX clusters (0)'. The 'Alarms' section shows a search bar and a table with columns 'Alarm name' and 'Status', indicating 'No custom alarms'. The 'DAX clusters' section shows a search bar and a table with columns 'Cluster name' and 'Status', indicating 'No clusters'. On the right, there are two panels: 'Create resources' with a 'Create table' button and 'What's new' with a 'Create DAX cluster' button. Below the main content, there is a 'Tables (0)' section with a search bar and a table with columns 'Name', 'Status', 'Partition key', 'Sort key', 'Indexes', 'Deletion protection', 'Read capacity mode', 'Write capacity mode', and 'Total size'. A message states 'You have no tables in this account in this AWS Region.' and a 'Create table' button is visible.

- **Activity 3.2: Create a DynamoDB table for storing registration details and book requests.**
 - Create Users table with partition key “Username” with type String and click on create tables.



The screenshot shows the 'Create table' form in the AWS DynamoDB console. The form is titled 'Create table' and includes a 'Table details' section. The 'Table name' field is labeled 'Users'. The 'Partition key' section shows 'Username' as the partition key with a 'String' data type. The 'Sort key - optional' section is currently empty, with a placeholder 'Enter the sort key name' and a 'String' data type. The form also includes a search bar at the top and a navigation bar with 'DynamoDB' and 'Tables' links.

☰

Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

Tags

Tags are pairs of keys and optional values, that you can assign to AWS resources. You can use tags to control access to your resources or track your AWS spending.

No tags are associated with the resource.

Add new tag

You can add 50 more tags.

Cancel
Create table

DynamoDB

Dashboard

Tables

Explore Items

PartiQL editor

Backups

Exports to S3

Imports from S3

Integrations New

The Users table was created successfully.
✕

DynamoDB > Tables

Tables (1) info

Any tag key

Any tag value

Actions
Delete
Create table

☐	Name ▲	Status ▼	Partition key ▼	Sort key ▼	Indexes ▼	Deletion protection ▼	Read capacity mode ▼	Write capacity mode ▼	Total size ▼
☐	Users	Active	email (S)	-	0	Off	Provisioned (5)	Provisioned (5)	0 bytes

- Follow the same steps to create an Orders table with Order_id as the primary key to store Order details.

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability.

String

▼

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

String

▼

1 to 255 characters and case sensitive.



Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

Tags

Tags are pairs of keys and optional values, that you can assign to AWS resources. You can use tags to control access to your resources or track your AWS spending.

No tags are associated with the resource.

You can add 50 more tags.

[Cancel](#)

Tables (2) [Info](#)

Find tables Any tag key Any tag value

☐	Name ▲	Status ▼	Partition key ▼	Sort key ▼	Indexes ▼	Replication Regions ▼	Deletion protecti
☐	Orders	Active	order_id (S)	-	0	0	Off
☐	Users	Active	username (S)	-	0	0	Off

Milestone 4: SNS Notification Setup

- **Activity 4.1: Create SNS topics for sending email notifications to users and library staff.**
 - In the AWS Console, search for SNS and navigate to the SNS Dashboard.

Search results for 'sns'

Services

Features

Resources **New**

Documentation

Knowledge articles

Marketplace

Blog posts

Events

Tutorials

Services [Show more ▶](#)


Simple Notification Service ☆
 SNS managed message topics for Pub/Sub


Route 53 Resolver
 Resolve DNS queries in your Amazon VPC and on-premises network.


Route 53 ☆
 Scalable DNS and Domain Name Registration


AWS End User Messaging ☆
 Engage your customers across multiple communication channels

Features [Show more ▶](#)

Events

 ElastiCache feature

SMS

 AWS End User Messaging feature

Hosted zones

 Route 53 feature

Amazon SNS

- Dashboard
- Topics
- Subscriptions
- ▼ Mobile
 - Push notifications
 - Text messaging (SMS)

New Feature
Amazon SNS now supports in-place message archiving and replay for FIFO topics. [Learn more](#)

Application Integration

Amazon Simple Notification Service

Pub/sub messaging for microservices and serverless applications.

Amazon SNS is a highly available, durable, secure, fully managed pub/sub messaging service that enables you to decouple microservices, distributed systems, and event-driven serverless applications. Amazon SNS provides topics for high-throughput, push-based, many-to-many messaging.

Create topic

Topic name

A topic is a message channel. When you publish a message to a topic, it fans out the message to all subscribed endpoints.

Next step

[Start with an overview](#)

Pricing

- Click on **Create Topic** and choose a name for the topic.

Amazon SNS

- Dashboard
- Topics**
- Subscriptions
- ▼ Mobile
 - Push notifications
 - Text messaging (SMS)

New Feature
Amazon SNS now supports in-place message archiving and replay for FIFO topics. [Learn more](#)

Amazon SNS > Topics

Topics (0)

Edit
Delete
Publish message
Create topic

Name	Type	ARN
No topics To get started, create a topic. Create topic		

- Choose Standard type for general notification use cases and Click on Create Topic.

[Amazon SNS](#) > [Topics](#) > [Create topic](#)

Create topic

Details

Type | [Info](#)

Topic type cannot be modified after topic is created

☐ **FIFO (first-in, first-out)**

- Strictly-preserved message ordering
- Exactly-once message delivery
- High throughput, up to 300 publishes/second
- Subscription protocols: SQS

☒ **Standard**

- Best-effort message ordering
- At-least once message delivery
- Highest throughput in publishes/second
- Subscription protocols: SQS, Lambda, HTTP, SMS, email, mobile application endpoints

Name

BookRequestNotifications

Maximum 256 characters. Can include alphanumeric characters, hyphens (-) and underscores (_).

Display name - *optional* | [Info](#)

To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

My Topic

Maximum 100 characters.

► Access policy - *optional* | [Info](#)

This policy defines who can access your topic. By default, only the topic owner can publish or subscribe to the topic.

► Data protection policy - *optional* | [Info](#)

This policy defines which sensitive data to monitor and to prevent from being exchanged via your topic.

► Delivery policy (HTTP/S) - *optional* | [Info](#)

The policy defines how Amazon SNS retries failed deliveries to HTTP/S endpoints. To modify the default settings, expand this section.

► Delivery status logging - *optional* | [Info](#)

These settings configure the logging of message delivery status to CloudWatch Logs.

► Tags - *optional*

A tag is a metadata label that you can assign to an Amazon SNS topic. Each tag consists of a key and an optional value. You can use tags to search and filter your topics and track your costs. [Learn more](#)

► Active tracing - *optional* | [Info](#)

Use AWS X-Ray active tracing for this topic to view its traces and service map in Amazon CloudWatch. Additional costs apply.

Cancel

Create topic

- Configure the SNS topic and note down the **Topic ARN**.

Amazon SNS

Dashboard
Topics
Subscriptions

▼ Mobile
Push notifications
Text messaging (SMS)

New Feature
Amazon SNS now supports in-place message archiving and replay for FIFO topics. [Learn more](#)

Topic: BookRequestNotifications created successfully.
You can create subscriptions and send messages to them from this topic.

Amazon SNS > Topics > BookRequestNotifications

BookRequestNotifications

Edit Delete Publish message

Details

Name BookRequestNotifications	Display name -
ARN arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications	Topic owner 557690616836
Type Standard	

Subscriptions Access policy Data protection policy Delivery policy (HTTP/S) Delivery status logging Encryption Tags Integrations

Subscriptions (0)

Search

Edit Delete Request confirmation Confirm subscription Create subscription

ID	Endpoint	Status	Protocol
No subscriptions found You don't have any subscriptions to this topic. Create subscription			

- **Activity 4.2: Subscribe users and staff to relevant SNS topics to receive real-time notifications when a book request is made.**

- Subscribe users (or admin staff) to this topic via Email. When a book request is made, notifications will be sent to the subscribed emails.

Amazon SNS > Subscriptions > Create subscription

Create subscription

Details

Topic ARN
arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications

Protocol
The type of endpoint to subscribe
Email

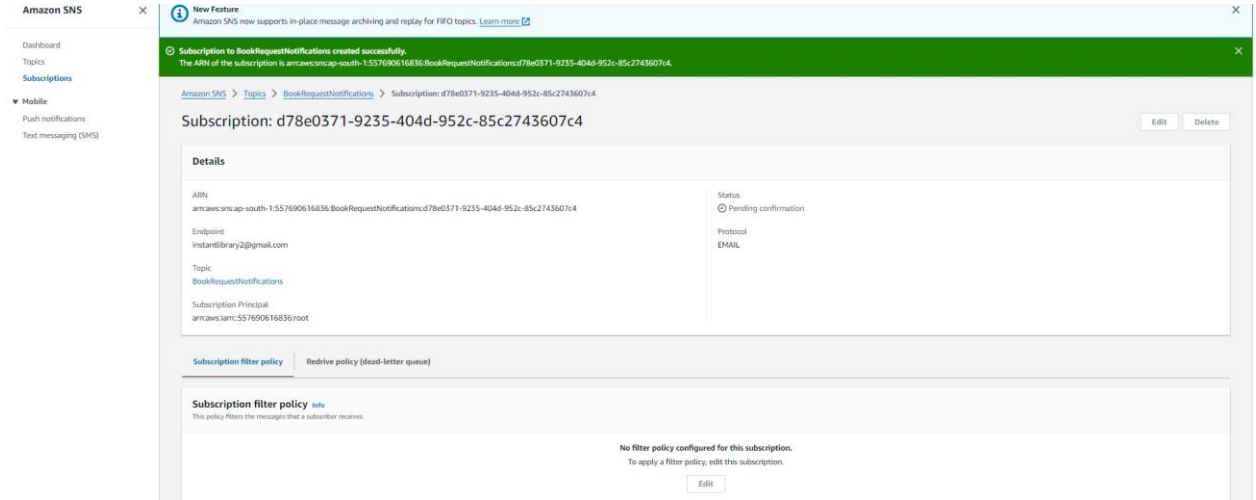
Endpoint
An email address that can receive notifications from Amazon SNS.
instantlibrary2@gmail.com

After your subscription is created, you must confirm it. [Info](#)

► **Subscription filter policy - optional** [Info](#)
This policy filters the messages that a subscriber receives.

► **Redrive policy (dead-letter queue) - optional** [Info](#)
Send undeliverable messages to a dead-letter queue.

Cancel [Create subscription](#)



Amazon SNS

New Feature
Amazon SNS now supports in-place message archiving and replay for FIFO topics. [Learn more](#)

Subscription to BookRequestNotifications created successfully.
The ARN of the subscription is `arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications:d78e0371-9235-404d-952c-85c2743607c4`.

Amazon SNS > Topics > BookRequestNotifications > Subscription: d78e0371-9235-404d-952c-85c2743607c4

Subscription: d78e0371-9235-404d-952c-85c2743607c4 [Edit] [Delete]

Details

ARN <code>arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications:d78e0371-9235-404d-952c-85c2743607c4</code>	Status Ⓢ Pending confirmation
Endpoint <code>instantlibrary2@gmail.com</code>	Protocol EMAIL
Topic BookRequestNotifications	
Subscription Principal <code>arn:aws:iam::557690616836:root</code>	

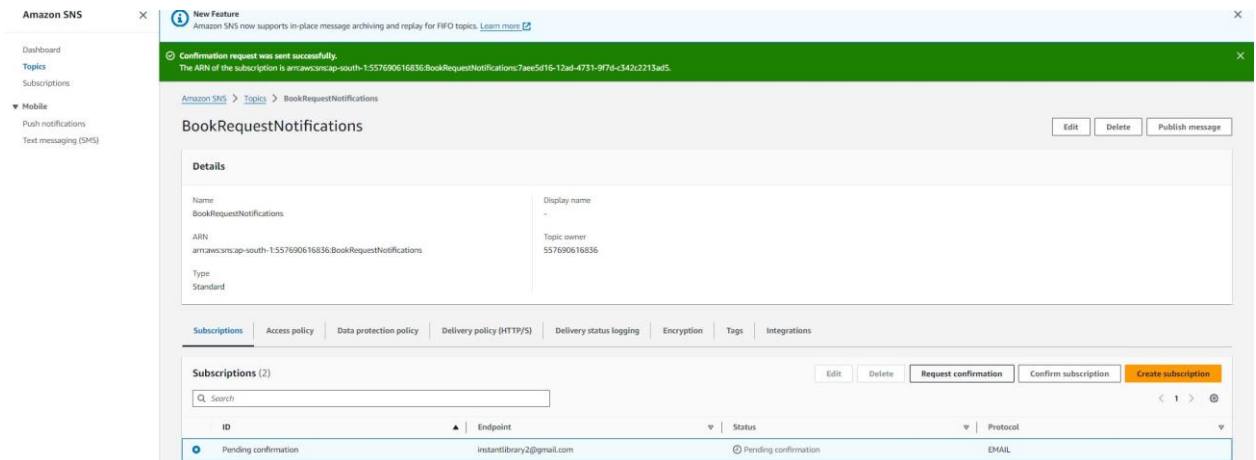
Subscription filter policy | **Redrive policy (dead-letter queue)**

Subscription filter policy info
This policy filters the messages that a subscriber receives.

No filter policy configured for this subscription.
To apply a filter policy, edit this subscription.

[Edit]

○ After subscription request for the mail confirmation



Amazon SNS

New Feature
Amazon SNS now supports in-place message archiving and replay for FIFO topics. [Learn more](#)

Confirmation request was sent successfully.
The ARN of the subscription is `arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications:7ae5f16-12ad-4731-97d4-34d22113ad5`.

Amazon SNS > Topics > BookRequestNotifications

BookRequestNotifications [Edit] [Delete] [Publish message]

Details

Name <code>BookRequestNotifications</code>	Display name -
ARN <code>arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications</code>	Topic owner <code>557690616836</code>
Type Standard	

Subscriptions | Access policy | Data protection policy | Delivery policy (HTTP/S) | Delivery status logging | Encryption | Tags | Integrations

Subscriptions (2) [Edit] [Delete] [Request confirmation] [Confirm subscription] [Create subscription]

Search

ID	Endpoint	Status	Protocol
Ⓢ Pending confirmation	<code>instantlibrary2@gmail.com</code>	Ⓢ Pending confirmation	EMAIL

○ Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail.

AWS Notification - Subscription Confirmation Inbox x

AWS Notifications <no-reply@sns.amazonaws.com>
to me ▼

9

You have chosen to subscribe to the topic:

arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):

[Confirm subscription](#)

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-opt-out](#)

AWS Notifications <no-reply@sns.amazonaws.com>
to me ▼

...

You have chosen to subscribe to the topic:

arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):

[Confirm subscription](#)

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-opt-out](#)



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications:d78e0371-9235-404d-952c-85c2743607c4

If it was not your intention to subscribe, [click here to unsubscribe](#).

- Successfully done with the SNS mail subscription and setup, now store the ARN link.

Amazon SNS > Topics > BookRequestNotifications

BookRequestNotifications Edit Delete Publish message

Details

Name BookRequestNotifications	Display name -
ARN arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications	Topic owner 557690616836
Type Standard	

Subscriptions | Access policy | Data protection policy | Delivery policy (HTTP/S) | Delivery status logging | Encryption | Tags | Integrations

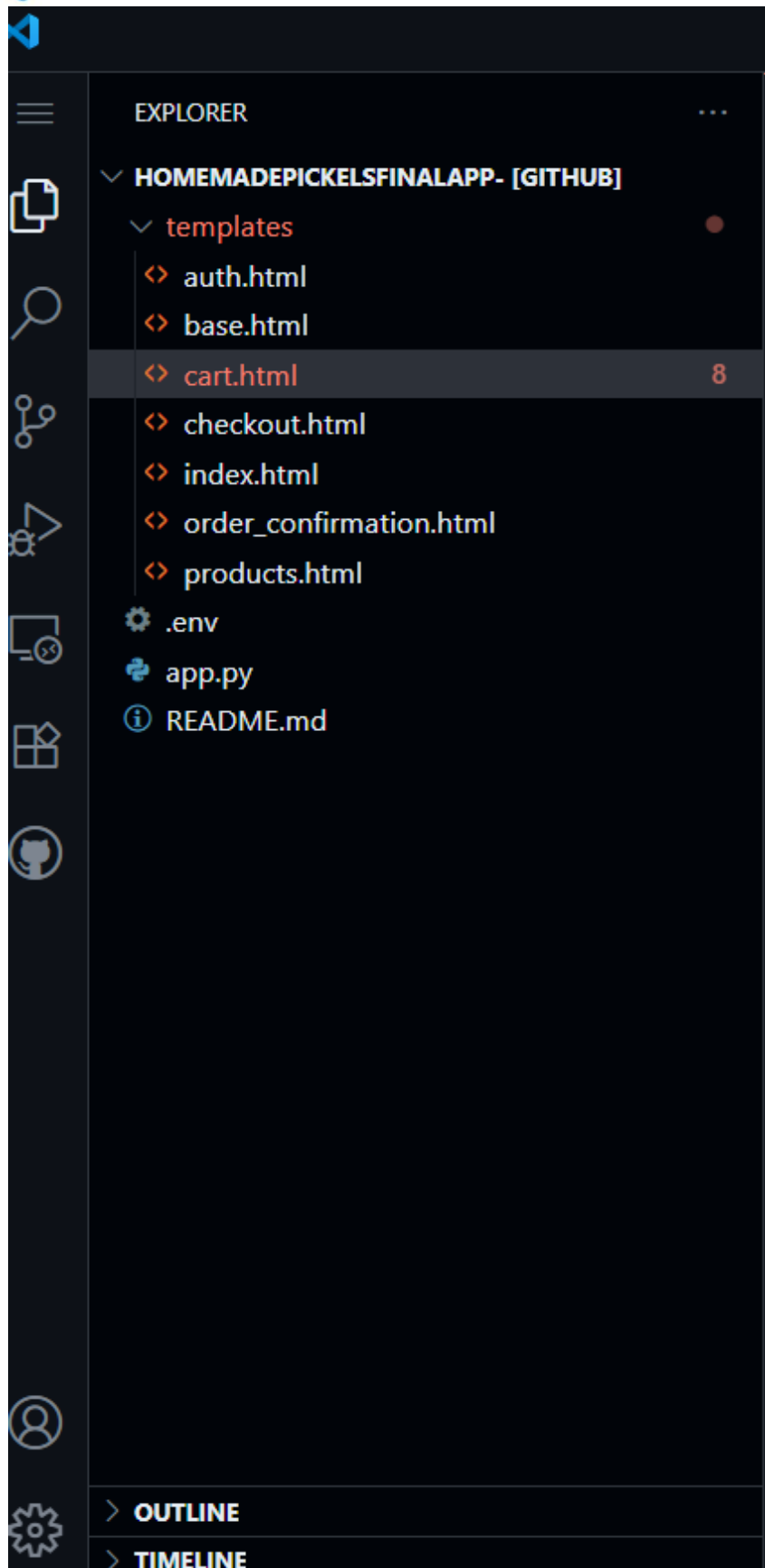
Subscriptions (2) Edit Delete Request confirmation Confirm subscription Create subscription

Q Search < 1 > ⓘ

ID	Endpoint	Status	Protocol
d7be0371-9235-4046-952c-85c3743607c4	instantlibrary2@gmail.com	Confirmed	EMAIL

Milestone 5: Backend Development and Application Setup

- **Activity 5.1: Develop the backend using Flask**
 - File Explorer Structure



Description: set up the Home Made Pickles & Snacks: Taste the Best

This project is a Flask-based e-commerce platform for selling pickles and snacks, supporting both local and AWS (DynamoDB, SNS) deployments. It includes an app.py file for routing and logic, a templates/ folder for HTML pages like index.html, auth.html, cart.html, and a static/ folder for CSS, JS, and images. Environment config is managed through .env to toggle AWS or local mode.

Description of the code :

- **Flask App Initialization**

```
pp.py > ...  
from flask import Flask, render_template, request, redirect, url_for, session, flash  
from werkzeug.security import generate_password_hash, check_password_hash  
from datetime import datetime  
import boto3  
import os  
import uuid  
from dotenv import load_dotenv
```

Description:

This project is a Flask-based web app for selling pickles and snacks, supporting both local and AWS deployments via DynamoDB and SNS. It includes app.py with routing logic, user authentication, and order management using Bcrypt for password hashing and Boto3 for database operations. The structure includes a templates/ folder for HTML pages and a static/ folder for assets, with .env handling configuration.

```
app = Flask(__name__)
```

Description: initialize the Flask application instance using Flask(__name__) to start building the web app.

- **DynamoDB Setup:**

```
USE_AWS = os.getenv('USE_AWS', 'False').lower() == 'true'
AWS_REGION_NAME = os.getenv('AWS_REGION_NAME', 'us-east-1')
USERS_TABLE = os.getenv('USERS_TABLE', 'Users')
ORDERS_TABLE = os.getenv('ORDERS_TABLE', 'Orders')

users = {}
orders = []

if USE_AWS:
    dynamodb = boto3.resource('dynamodb', region_name=AWS_REGION_NAME)
    users_table = dynamodb.Table(USERS_TABLE)
    orders_table = dynamodb.Table(ORDERS_TABLE)
```

Description: initialize the DynamoDB resource for the ap-south-1 region and set up access to the Users and Requests tables for storing user details and book requests.

- **SNS Connection**

```
# SNS Topic ARN (create the SNS topic in AWS and provide the ARN here)
sns = boto3.client('sns', region_name='ap-south-1')
sns_topic_arn = 'arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications'

# Email settings (for sending emails)
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587
SENDER_EMAIL = "instantlibrary2@gmail.com"
SENDER_PASSWORD = "luut dsih nyvq dgzv" # Your app password

# Function to send email
def send_email(to_email, subject, body):
    msg = MIMEText(body, 'plain')
    msg['From'] = SENDER_EMAIL
    msg['To'] = to_email
    msg['Subject'] = subject
    msg.attach(MIMEText(body, 'plain'))

    try:
        server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
        server.starttls()
        server.login(SENDER_EMAIL, SENDER_PASSWORD)
        text = msg.as_string()
        server.sendmail(SENDER_EMAIL, to_email, text)
        server.quit()
        print("Email sent successfully")
    except Exception as e:
        print(f"Failed to send email: {e}")
```

Description: Configure **SNS** to send notifications when a book request is submitted. Paste your stored ARN link in the sns_topic_arn space, along with the region_name where the SNS topic is created. Also, specify the chosen email service in SMTP_SERVER

(e.g., Gmail, Yahoo, etc.) and enter the subscribed email in the SENDER_EMAIL section. Create an 'App password' for the email ID and store it in the SENDER_PASSWORD section. • **Routes for Web Pages**

- **Home Route:**

```
@app.route('/')  
def home():  
    return render_template('index.html')
```

Description: define the home route / to automatically redirect users to the register page when they access the base URL.

- **Register Route:**

```

def register():
    username = request.form['username'].strip()
    email = request.form['email']
    password = request.form['password']

    if USE_AWS:
        try:
            response = users_table.get_item(Key={'username': username})
            if 'Item' in response:
                flash('Username already exists!')
                return redirect(url_for('auth'))
            users_table.put_item(Item={
                'username': username,
                'email': email,
                'password': generate_password_hash(password)
            })
        except Exception as e:
            flash('Error with AWS: ' + str(e))
            return redirect(url_for('auth'))
    else:
        if username in users:
            flash('Username already exists!')
            return redirect(url_for('auth'))
        users[username] = {
            'email': email,
            'password': generate_password_hash(password),
            'orders': []
        }

    session['username'] = username
    flash('Registration successful!')
    return redirect(url_for('show_products'))

```

• **Description:** define /register route to validate registration form fields, hash the user password using Bcrypt, store the new user in DynamoDB with a login count, and send an SNS notification on successful registration

- **login Route (GET/POST):**

```
def login():
    username = request.form['username'].strip()
    password = request.form['password']

    user_data = None
    if USE_AWS:
        response = users_table.get_item(Key={'username': username})
        user_data = response.get('Item')
    else:
        user_data = users.get(username)

    if user_data and check_password_hash(user_data['password'], password):
        session['username'] = username
        return redirect(url_for('show_products'))

    flash('Invalid username or password!')
    return redirect(url_for('auth'))
```

Description: define /login route to validate user credentials against DynamoDB, check the password using Bcrypt, update the login count on successful authentication, and redirect users to the home page


```
def show_products():
    if 'username' not in session:
        return redirect(url_for('auth'))
    return render_template('products.html', products=products)

@app.route('/add_to_cart/<int:product_id>')
def add_to_cart(product_id):
    if 'username' not in session:
        return redirect(url_for('auth'))

    cart = session.get('cart', {})
    cart[str(product_id)] = cart.get(str(product_id), 0) + 1
    session['cart'] = cart

    flash(f'{products[product_id]["name"]} added to cart!')
    return redirect(url_for('show_products'))

@app.route('/cart')
def cart():
    if 'username' not in session:
        return redirect(url_for('auth'))

    cart = session.get('cart', {})
    cart_items = []
    total = 0

    for pid, qty in cart.items():
        product = products[int(pid)]
        subtotal = product['price'] * qty
        cart_items.append({
```

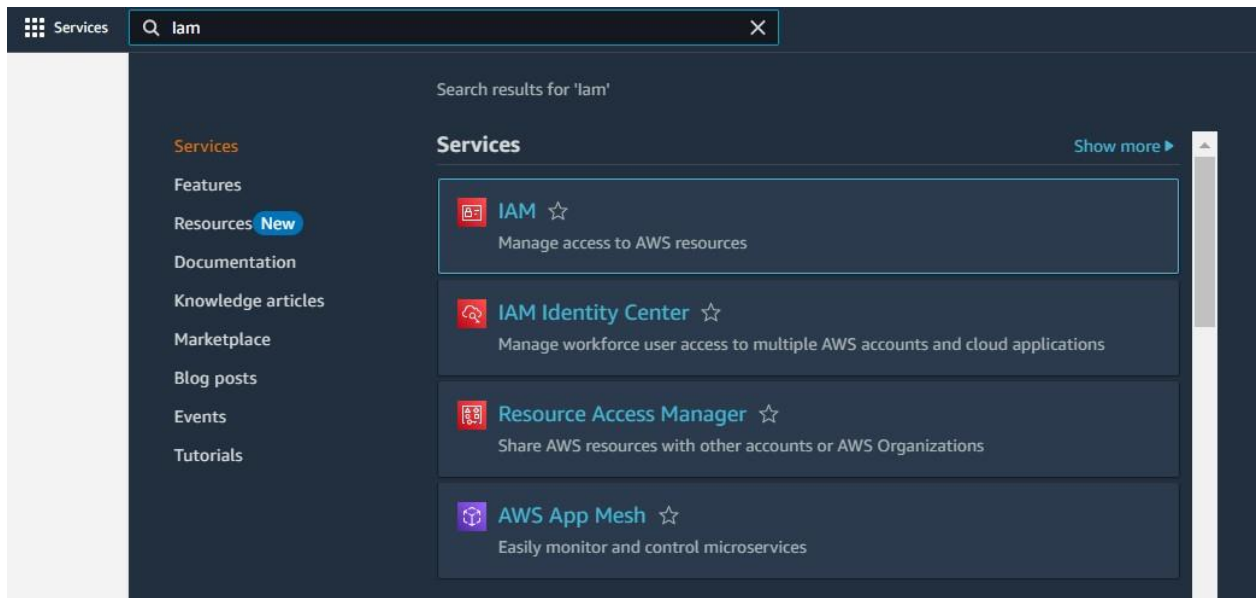
Deployment Code:

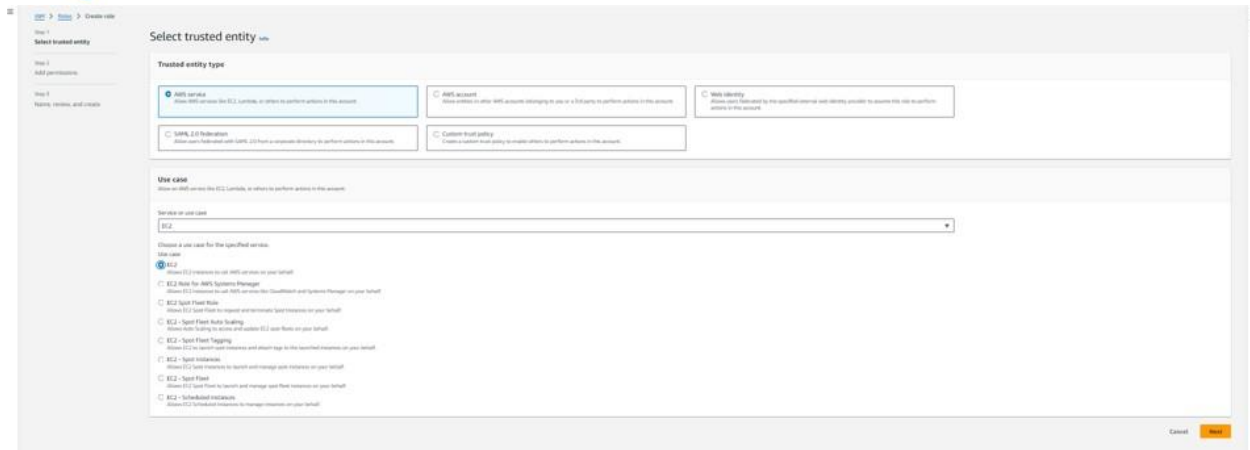
```
if __name__ == "__main__":  
    app.run(host='0.0.0.0', port=80, debug=True)
```

Description: start the Flask server to listen on all network interfaces (0.0.0.0) at port 80 with debug mode enabled for development and testing.

Milestone 6: IAM Role Setup

- **Activity 6.1: Create IAM Role.**
 - In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB and SNS.





Select trusted entity

Trusted entity type

- ☒ **AWS service**
Allows AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allows another AWS account (belonging to you or a third party) to perform actions in this account.
- ☐ **Web identity**
Allows users (managed by this provider) to assume this role to perform actions in this account.
- ☐ **OpenID Connect**
Allows OpenID Connect providers to perform actions in this account.
- ☐ **Custom trust policy**
Creates a custom trust policy to enable actions to perform actions in this account.

User case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

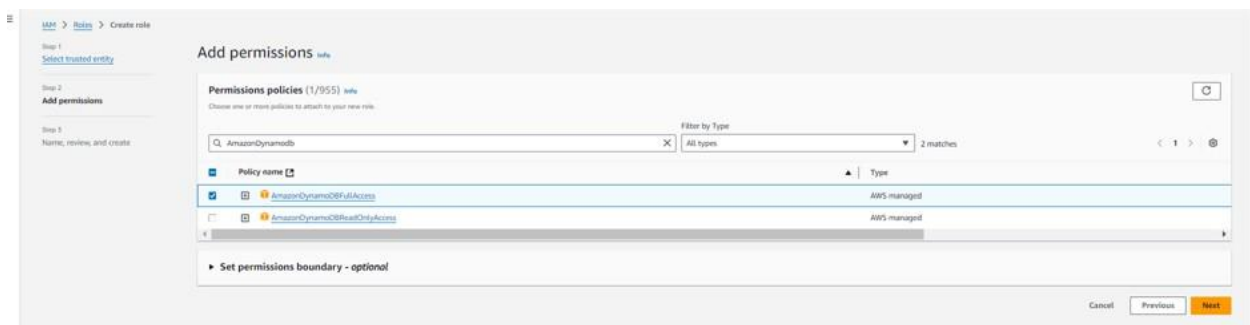
Service or user case
EC2

Choose a use case for the specified service.

Use case

- ☒ **EC2**
Allows EC2 instances to call AWS services on your behalf.
- ☐ **EC2 Role for AWS Systems Manager**
Allows EC2 instances to call AWS services like CloudWatch and Systems Manager on your behalf.
- ☐ **EC2 Spot Fleet Role**
Allows EC2 Spot Fleet to request and terminate Spot instances on your behalf.
- ☐ **EC2 - Spot Fleet Auto Scaling**
Allows auto scaling to create and update EC2 spot fleets on your behalf.
- ☐ **EC2 - Spot Fleet Tagging**
Allows EC2 to search and tag instances and attach tags to the specified instances on your behalf.
- ☐ **EC2 - Spot Instance**
Allows EC2 to search and tag instances and attach tags to the specified instances on your behalf.
- ☐ **EC2 - Scheduled instances**
Allows EC2 Scheduled instances to manage instances on your behalf.

Cancel **Next**



Add permissions

Permissions policies (1/955)

Choose one or more policies to attach to your new role.

Filter by Type: All types 2 matches

Policy name	Type
☒ AmazonDynamoDBFullAccess	AWS managed
☐ AmazonDynamoDBReadOnlyAccess	AWS managed

▶ Set permissions boundary - optional

Cancel Previous **Next**

● Activity 6.2: Attach Policies.

Attach the following policies to the role:

- **AmazonDynamoDBFullAccess:** Allows EC2 to perform read/write operations on DynamoDB.
- **AmazonSNSFullAccess:** Grants EC2 the ability to send notifications via SNS.

100 IAM > Roles > Create role

Step 1
[Select trusted entity](#)

Step 2
Add permissions

Step 3
Name, review, and create

Add permissions info

Permissions policies (2/955) info

Choose one or more policies to attach to your new role.

Filter by type

Q: srs X All types 5 matches (1) 0

Policy name	Type
AmazonS3FullAccess	AWS managed
AmazonS3OutpostsFullAccess	AWS managed
AmazonS3OutpostsReadOnlyAccess	AWS managed
AmazonS3OutpostsReadOnlyAccess	AWS managed
AmazonS3OutpostsReadOnlyAccess	AWS managed
AmazonS3OutpostsReadOnlyAccess	AWS managed

► Set permissions boundary - optional

Cancel Previous **Next**

100 IAM > Roles > Create role

Step 1
[Select trusted entity](#)

Step 2
[Add permissions](#)

Step 3
Name, review, and create

Name, review, and create

Role details

Role name:

Description:

Permissions:

Step 1: Select trusted entities info

Trust policy

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": "*"
    }
  ]
}

```

Step 2: Add permissions info

Permissions policy summary

Policy name	Type	Attached as
AmazonS3FullAccess	AWS managed	Permissions policy
AmazonS3OutpostsFullAccess	AWS managed	Permissions policy

Step 3: Add tags info

Add tags - optional info

Tag is key-value pair that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

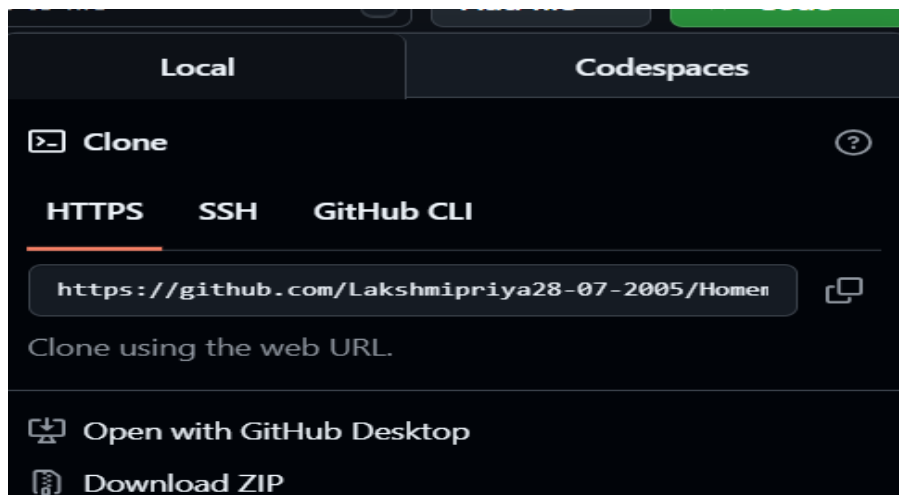
You can add up to 50 more tags.

Cancel Previous **Create role**

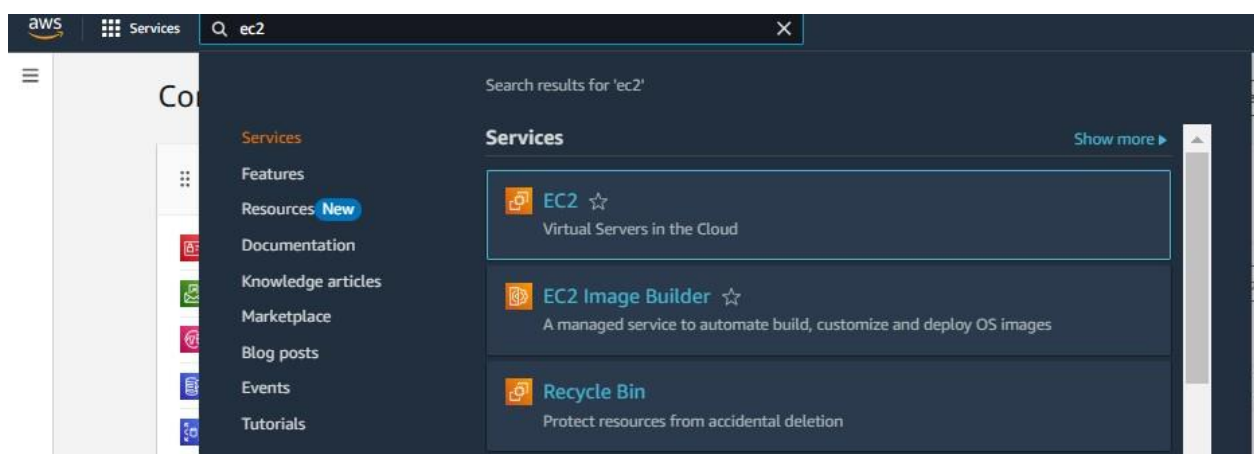
Milestone 7: EC2 Instance Setup

- **Note: Load your Flask app and Html files into GitHub repository.**

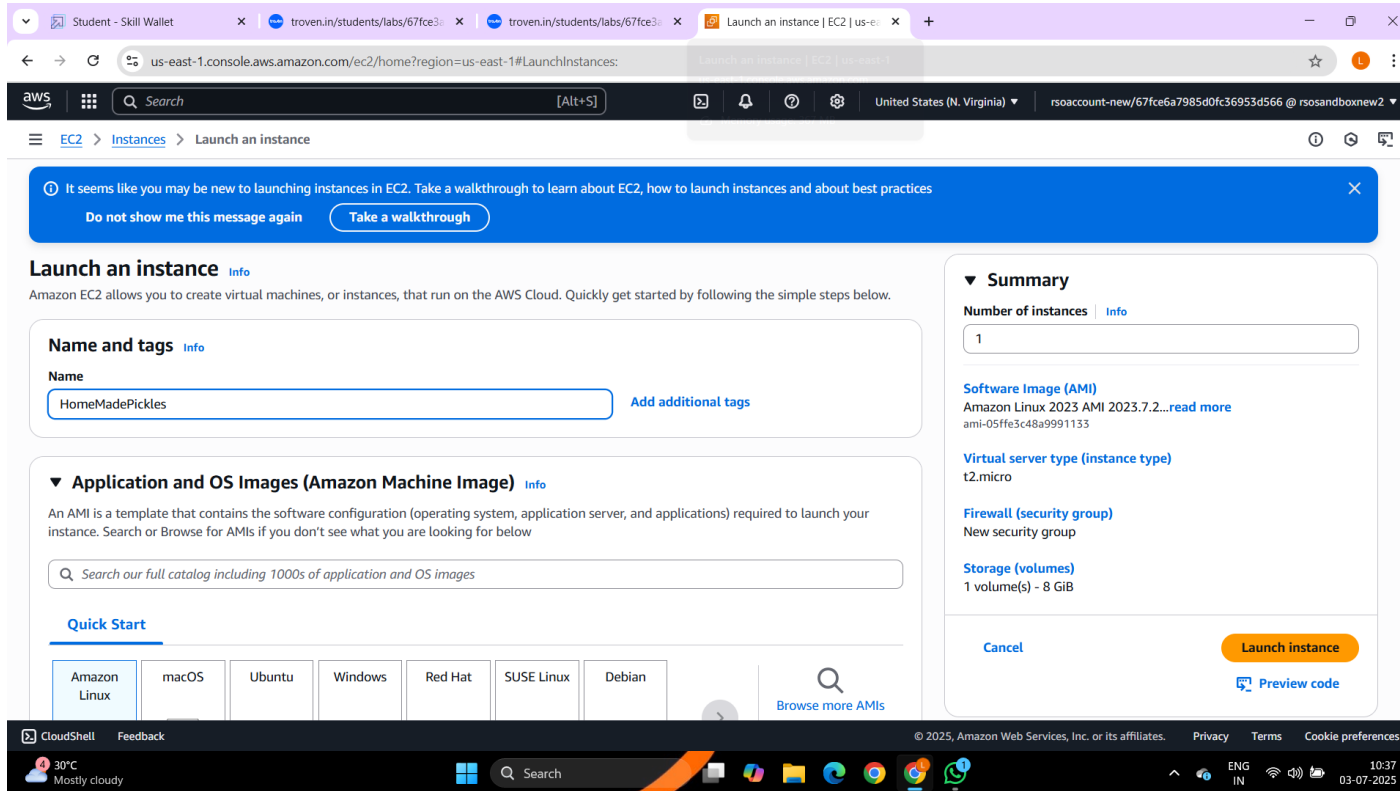
 Lakshmipriya28-07-2005	Add files via upload	b149d15 · 23 minutes ago	 11 Commits
 templates	Add files via upload	23 minutes ago	
 .env	Add files via upload	23 minutes ago	
 README.md	Add files via upload	Initial commit	last week
 app.py	Add files via upload	23 minutes ago	



- **Activity 7.1: Launch an EC2 instance to host the Flask application.**
- **Launch EC2 Instance**
 - In the AWS Console, navigate to EC2 and launch a new instance.



- Click on Launch instance to launch EC2 instance



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:

Launch an instance | EC2 | us-east-1

aws Search [Alt+S] United States (N. Virginia) rsoaccount-new/67fce6a7985d0fc36953d566 @ rsoandboxnew2

EC2 > Instances > Launch an instance

It seems like you may be new to launching instances in EC2. Take a walkthrough to learn about EC2, how to launch instances and about best practices

Do not show me this message again Take a walkthrough

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name

HomeMadePickles [Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) Info

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Search our full catalog including 1000s of application and OS images

Quick Start

Amazon Linux macOS Ubuntu Windows Red Hat SUSE Linux Debian [Browse more AMIs](#)

▼ Summary

Number of instances Info

1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.7.2...[read more](#)
ami-05ffe3c48a9991133

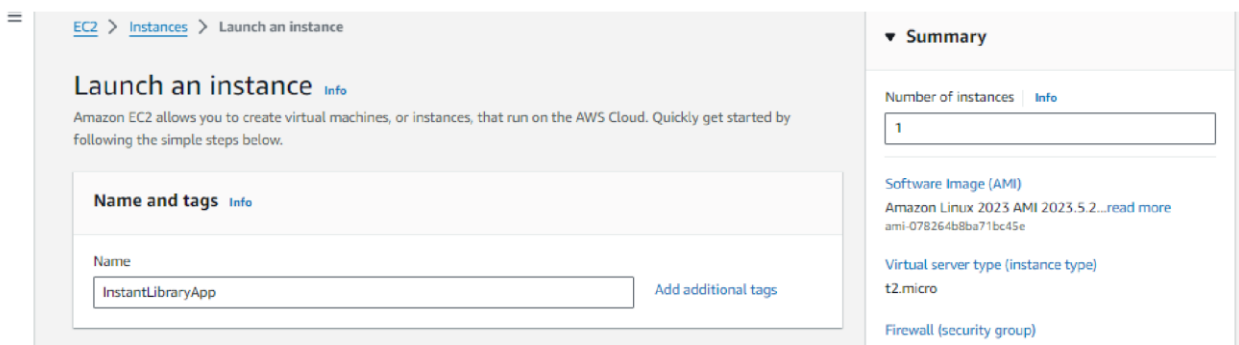
Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Cancel [Launch instance](#) [Preview code](#)

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences 30°C Mostly cloudy 10:37 03-07-2025



EC2 > Instances > Launch an instance

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name

InstantLibraryApp [Add additional tags](#)

▼ Summary

Number of instances Info

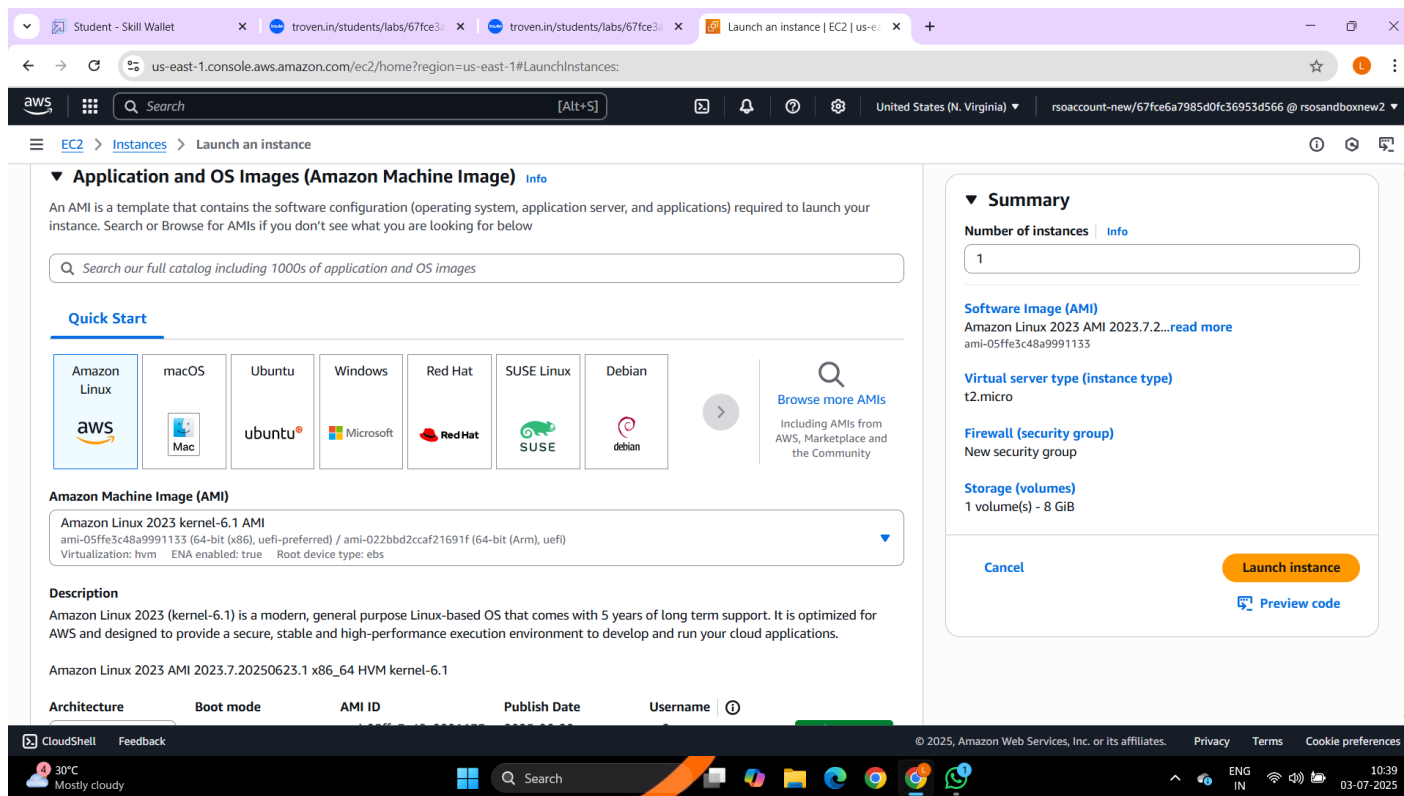
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.5.2...[read more](#)
ami-078264b8ba71bc45e

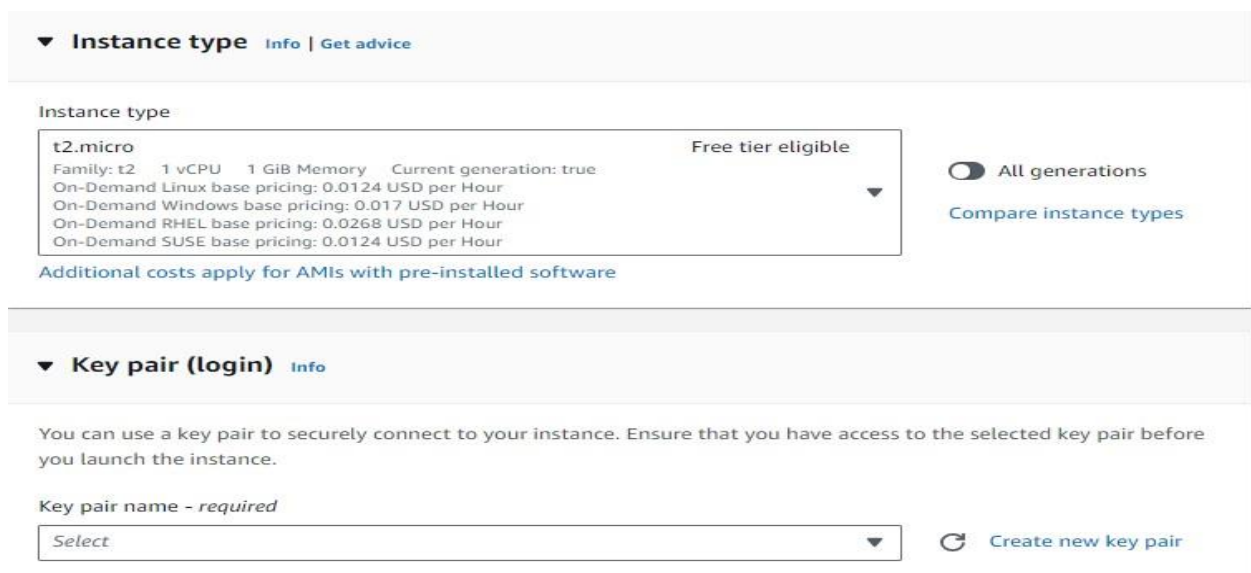
Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

- Choose Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instance type (free-tier eligible).



- Create and download the key pair for Server access.



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:

EC2 > Instances > Launch an instance

Instance type Info | Get advice

Instance type

t2.micro
Family: t2 1 vCPU 1 GiB Memory Current generation: true

Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

Select

Network settings Info

Network Info

vpc-022eb65325271b1da

Subnet Info

No preference (Default subnet in any availability zone)

Auto-assign public IP Info

Enable

Firewall (security groups) Info

Create key pair

Key pair name

Key pairs allow you to connect to your instance securely.

homemade

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA
RSA encrypted private and public key pair

☐ ED25519
ED25519 encrypted private and public key pair

Private key file format

☒ .pem
For use with OpenSSH

☐ .ppk
For use with PuTTY

When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn more](#)

Cancel Create key pair

Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.7.2...read more

ami-05ffe3c48a9991133

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel Launch instance Preview code

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

30°C Mostly cloudy

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:

EC2 > Instances > Launch an instance

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.7.20250623.1 x86_64 HVM kernel-6.1

Architecture	Boot mode	AMI ID	Publish Date	Username	
64-bit (x86)	uefi-preferred	ami-05ffe3c48a9991133	2025-06-20	ec2-user	Verified provider

Instance type Info | Get advice

Instance type

t2.micro
Family: t2 1 vCPU 1 GiB Memory Current generation: true

All generations

Compare instance types

Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

Select

Create new key pair

Network settings Info

Edit

Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.7.2...read more

ami-05ffe3c48a9991133

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel Launch instance Preview code

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

30°C Mostly cloudy

- **Activity 7.2: Configure security groups for HTTP, and SSH access.**

▼ Network settings

Info

VPC - required

Info

vpc-03cdc7b6f19dd7211

(default) ▼

172.31.0.0/16

↻

Subnet

Info

No preference

▼

↻

Create new subnet

↗

Auto-assign public IP

Info

Enable

▼

Additional charges apply when outside of free tier allowance

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

Security group name - required

launch-wizard

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-./()#,@[]+=&{}!\$*

Description - required

Info

launch-wizard created 2024-10-13T17:49:56.622Z

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 0.0.0.0/0)

Remove

Type

Info

ssh

▼

Source type

Info

Anywhere

▼

Protocol

Info

TCP

Port range

Info

22

Source

Info

Add CIDR, prefix list or security

0.0.0.0/0

✕

Description - optional

Info

e.g. SSH for admin desktop

▼ Security group rule 2 (TCP, 80, 0.0.0.0/0)

Remove

Type

Info

HTTP

▼

Source type

Info

Custom

▼

Protocol

Info

TCP

Port range

Info

80

Source

Info

Add CIDR, prefix list or security

0.0.0.0/0

✕

Description - optional

Info

e.g. SSH for admin desktop

▼ Security group rule 3 (TCP, 5000, 0.0.0.0/0)

Remove

Type

Info

Custom TCP

▼

Source type

Info

Custom

▼

Protocol

Info

TCP

Port range

Info

5000

Source

Info

Add CIDR, prefix list or security

0.0.0.0/0

✕

Description - optional

Info

e.g. SSH for admin desktop

Add security group rule

EC2 > ... > Launch an Instance

Success
Successfully initiated launch of Instance i-001861022fbcac2993

► Launch log

Next Steps

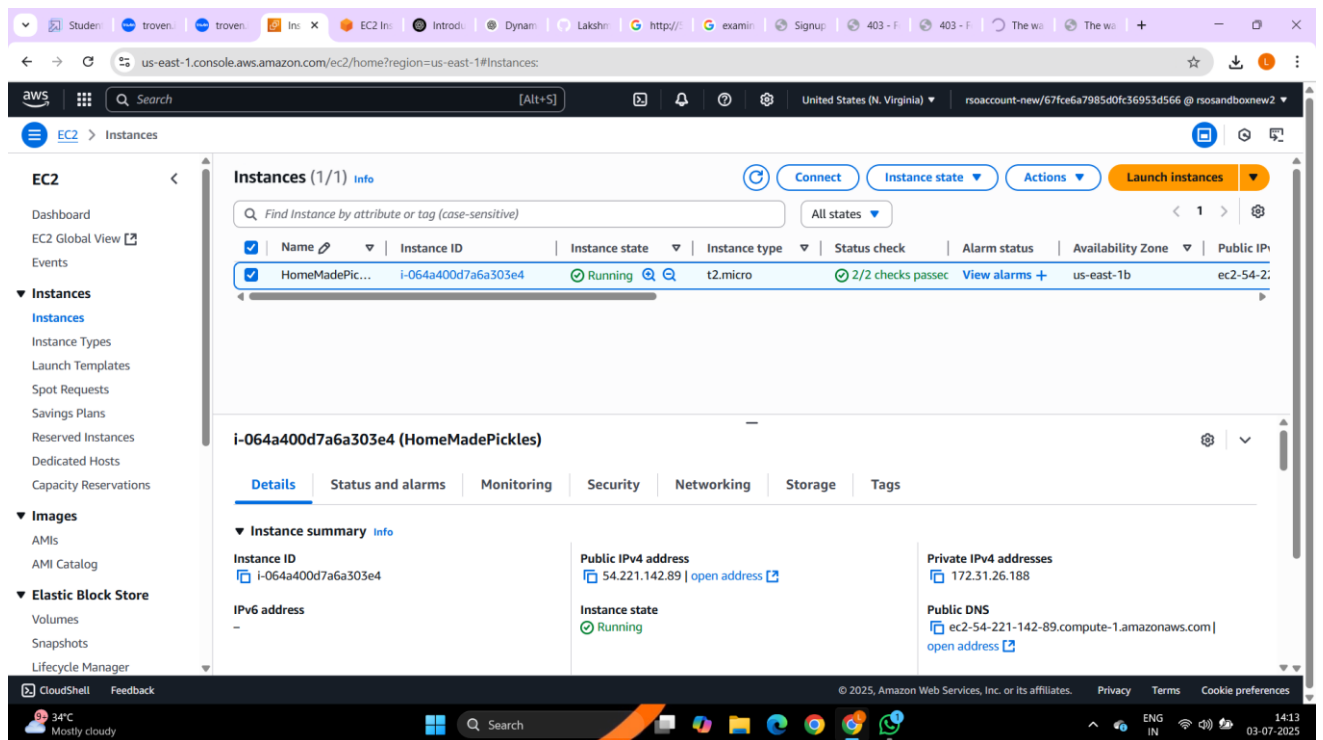
Q: What would you like to do next with this instance, for example "create alarm" or "create backup"

1 2 3 4 >

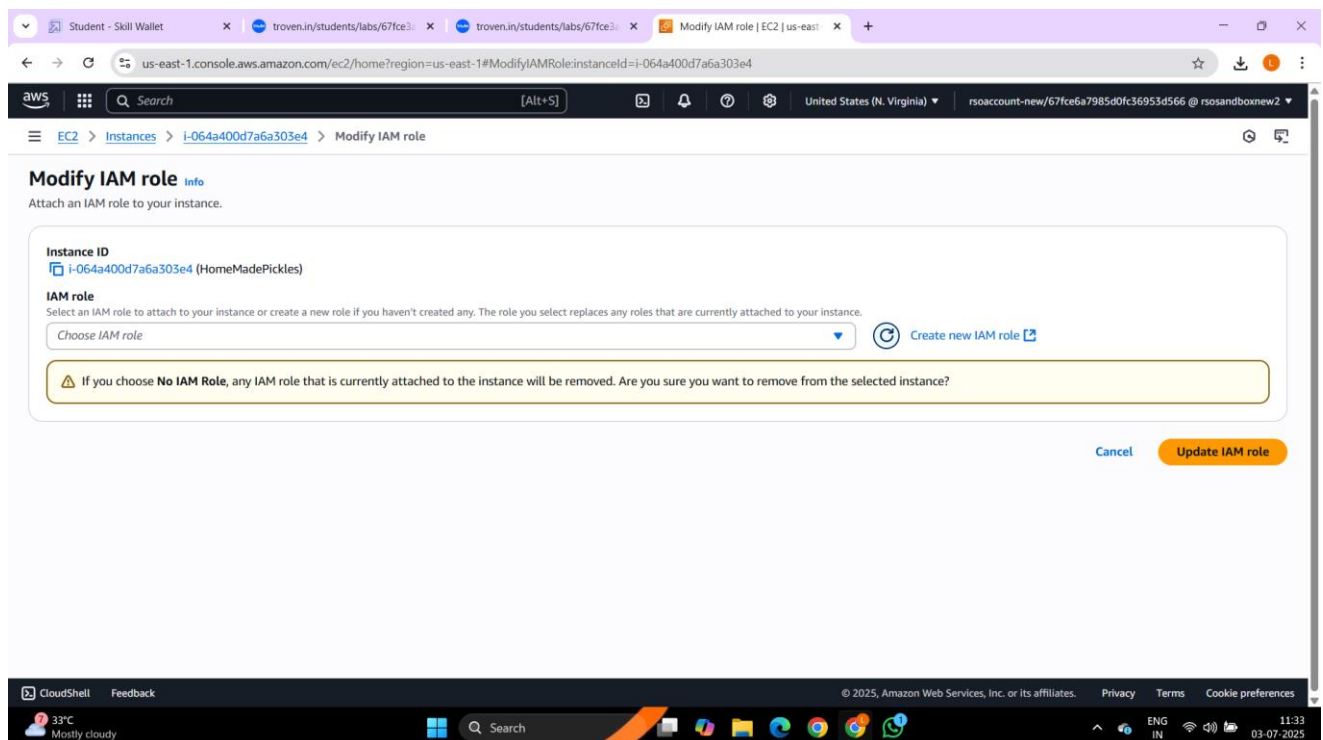
Create billing and free tier usage alerts To manage costs and avoid surprise bills, set up email notifications for billing and free tier usage thresholds. Create billing alerts	Connect to your instance Once your instance is running, log into it from your local computer. Connect to instance Learn more	Connect an RDS database Configure the connection between an EC2 instance and a database to allow traffic flow between them. Connect an RDS database Create a new RDS database Learn more	Create EBS snapshot policy Create a policy that automates the creation, retention, and deletion of EBS snapshots. Create EBS snapshot policy	Manage detailed monitoring Enable or disable detailed monitoring for the instance. If you enable detailed monitoring, the Amazon EC2 console displays monitoring graphs with a 1-minute period. Manage detailed monitoring	Create Load Balancer Create an application, network gateway or classic Elastic Load Balancer. Create Load Balancer
Create AWS budget AWS Budgets allows you to create budgets, forecast spend, and take action on your costs and usage from a single location. Create AWS budget	Manage CloudWatch alarms Create or update Amazon CloudWatch alarms for the instance. Manage CloudWatch alarms	Disaster recovery for your instances Recover the instances you just launched into a different Availability Zone or a different Region using AWS Elastic Disaster Recovery (DRS). Disaster recovery for your instances	Monitor for suspicious runtime activities Amazon GuardDuty enables you to continuously monitor for malicious runtime activity and unauthorized behavior, with near real-time visibility into on-host activities occurring across your Amazon EC2 workload. Monitor for suspicious runtime activities	Get instance screenshot Capture a screenshot from the instance and view it as an image. This is useful for troubleshooting an unreachable instance. Get instance screenshot	Get system log View the instance's system log to troubleshoot issues. Get system log

[View all instances](#)

- To connect to EC2 using **EC2 Instance Connect**, start by ensuring that an **IAM role** is attached to your EC2 instance. You can do this by selecting your instance, clicking on **Actions**, then navigating to **Security** and selecting **Modify IAM Role** to attach the appropriate role. After the IAM role is connected, navigate to the **EC2** section in the **AWS Management Console**. Select the **EC2 instance** you wish to connect to. At the top of the **EC2 Dashboard**, click the **Connect** button. From the connection methods presented, choose **EC2 Instance Connect**. Finally, click **Connect** again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.



The screenshot shows the AWS Management Console for the us-east-1 region. The left sidebar contains navigation options: EC2, Dashboard, EC2 Global View, Events, Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, Images, AMIs, AMI Catalog, Elastic Block Store, Volumes, Snapshots, and Lifecycle Manager. The main content area displays the 'Instances (1/1)' page. A table lists one instance: 'HomeMadePic...' with ID 'i-064a400d7a6a303e4', state 'Running', type 't2.micro', and availability zone 'us-east-1b'. Below the table, the 'Details' tab for instance 'i-064a400d7a6a303e4 (HomeMadePickles)' is selected. It shows the instance summary with fields: Instance ID (i-064a400d7a6a303e4), Public IPv4 address (54.221.142.89), Private IPv4 addresses (172.31.26.188), Instance state (Running), and Public DNS (ec2-54-221-142-89.compute-1.amazonaws.com).



The screenshot shows the 'Modify IAM role' page in the AWS Management Console. The breadcrumb trail is 'EC2 > Instances > i-064a400d7a6a303e4 > Modify IAM role'. The page title is 'Modify IAM role' with a sub-header 'Attach an IAM role to your instance.' Below this, the 'Instance ID' is 'i-064a400d7a6a303e4 (HomeMadePickles)'. The 'IAM role' section has a dropdown menu labeled 'Choose IAM role' and a 'Create new IAM role' button. A warning message states: 'If you choose No IAM Role, any IAM role that is currently attached to the instance will be removed. Are you sure you want to remove from the selected instance?'. At the bottom right, there are 'Cancel' and 'Update IAM role' buttons.

- Now connect the EC2 with the files

Connect to instance Info

Connect to your instance i-001861022fbcac290 (InstantLibraryApp) using any of these options

EC2 Instance Connect
Session Manager
SSH client
EC2 serial console


Port 22 (SSH) is open to all IPv4 addresses
 Port 22 (SSH) is currently open to all IPv4 addresses, indicated by 0.0.0.0/0 in the inbound rule in [your security group](#). For increased security, consider restricting access to only the EC2 Instance Connect service IP addresses for your Region: 13.233.177.0/29. [Learn more.](#)

Instance ID
 i-001861022fbcac290 (InstantLibraryApp)

Connection Type

☒ **Connect using EC2 Instance Connect**
 Connect using the EC2 Instance Connect browser-based client, with a public IPv4 or IPv6 address.

☐ **Connect using EC2 Instance Connect Endpoint**
 Connect using the EC2 Instance Connect browser-based client, with a private IPv4 address and a VPC endpoint.

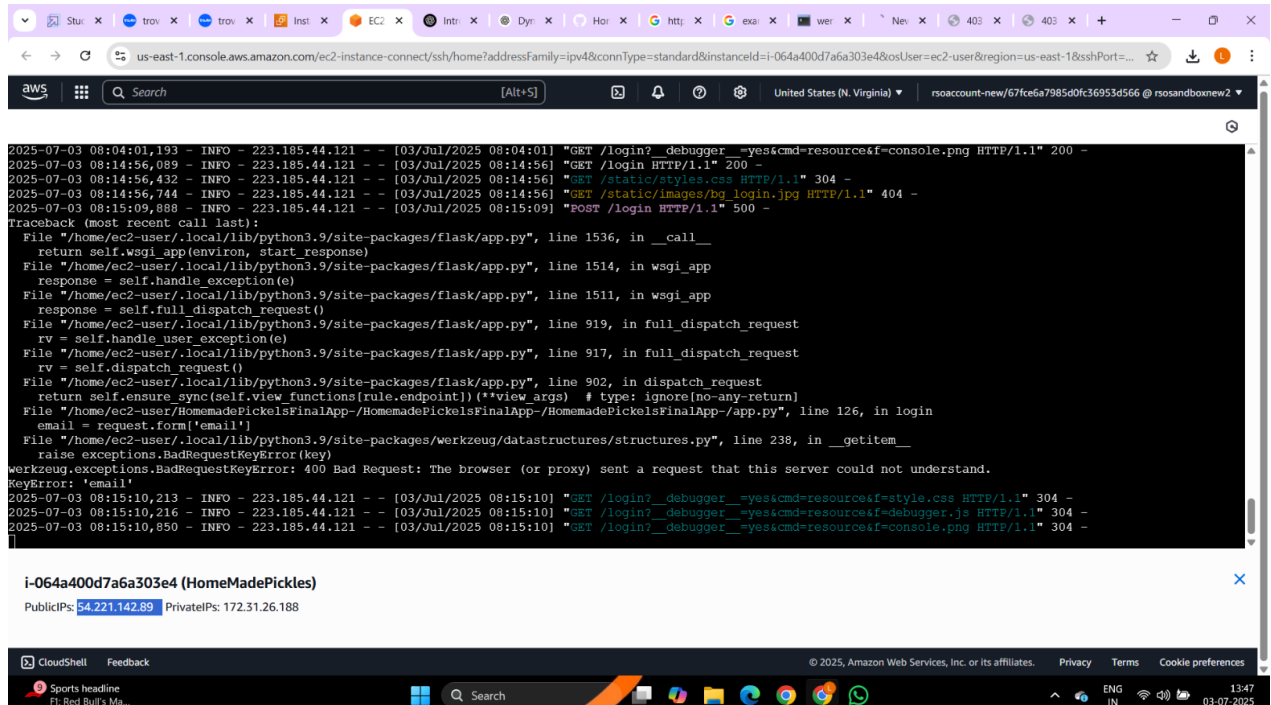
☒ **Public IPv4 address**
 13.200.229.59

☐ **IPv6 address**
 -

Username
 Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user.

 **Note:** In most cases, the default username, ec2-user, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Cancel Connect



The screenshot shows the AWS Management Console interface for an EC2 Instance Connect session. The top navigation bar includes the AWS logo, search bar, and account information (United States (N. Virginia), rsaccount-new/67fce6a7985d0fc36953d566 @ rsosandboxnew2). The main content area displays the session logs for the instance i-064a400d7a6a303e4 (HomeMadePickles). The logs show a series of HTTP requests and responses, including a 400 Bad Request error. Below the logs, the instance details are shown, including the Public IP address 54.221.142.89 and Private IP address 172.31.26.188. The bottom of the screen shows the Windows taskbar with various application icons and the system clock (13:47, 03-07-2025).

```

2025-07-03 08:04:01,193 - INFO - 223.185.44.121 - - [03/Jul/2025 08:04:01] "GET /login?_debugger__yes&cmd=resource&f=console.png HTTP/1.1" 200 -
2025-07-03 08:14:56,089 - INFO - 223.185.44.121 - - [03/Jul/2025 08:14:56] "GET /login HTTP/1.1" 200 -
2025-07-03 08:14:56,432 - INFO - 223.185.44.121 - - [03/Jul/2025 08:14:56] "GET /static/styles.css HTTP/1.1" 304 -
2025-07-03 08:14:56,744 - INFO - 223.185.44.121 - - [03/Jul/2025 08:14:56] "GET /static/images/bg_login.jpg HTTP/1.1" 404 -
2025-07-03 08:15:09,488 - INFO - 223.185.44.121 - - [03/Jul/2025 08:15:09] "POST /login HTTP/1.1" 500 -
Traceback (most recent call last):
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
  File "/home/ec2-user/HomemadePicklesFinalApp-/HomemadePicklesFinalApp-/HomemadePicklesFinalApp-/app.py", line 126, in login
    email = request.form['email']
  File "/home/ec2-user/.local/lib/python3.9/site-packages/werkzeug/datastructures/structures.py", line 238, in __getitem__
    raise exceptions.BadRequestKeyError(key)
werkzeug.exceptions.BadRequestKeyError: 400 Bad Request: The browser (or proxy) sent a request that this server could not understand.
KeyError: 'email'
2025-07-03 08:15:10,213 - INFO - 223.185.44.121 - - [03/Jul/2025 08:15:10] "GET /login?_debugger__yes&cmd=resource&f=style.css HTTP/1.1" 304 -
2025-07-03 08:15:10,216 - INFO - 223.185.44.121 - - [03/Jul/2025 08:15:10] "GET /login?_debugger__yes&cmd=resource&f=debugger.js HTTP/1.1" 304 -
2025-07-03 08:15:10,850 - INFO - 223.185.44.121 - - [03/Jul/2025 08:15:10] "GET /login?_debugger__yes&cmd=resource&f=console.png HTTP/1.1" 304 -
  
```

i-064a400d7a6a303e4 (HomeMadePickles)

PublicIPs: 54.221.142.89 PrivateIPs: 172.31.26.188

Milestone 8: Deployment on EC2

Activity 8.1: Install Software on the EC2 Instance

Install Python3, Flask, and Git:

On Amazon Linux 2:

```
sudo yum update -y
```

```
sudo yum install python3 git
```

```
sudo pip3 install flask boto3
```

Verify Installations:

```
flask --version
```

```
git --version
```

Activity 8.2: Clone Your Flask Project from GitHub:

Run: `git clone:`<https://github.com/kilarukusuma-12/Homemadepicklesandsnacks.git>

Note: change your-github-username and your-repository-name with your credentials

here: `'https://github.com/SAIGOKUL45977/Homemade-Pickles-Website.git'`

This will download your project to the EC2 instance.

To navigate to the project directory, run the following command:

```
cd "Homemade-Pickles-Website"
```

Create a Virtual Environment:

```
python3 -m venv venv
```

```
source venv/bin/activate
```

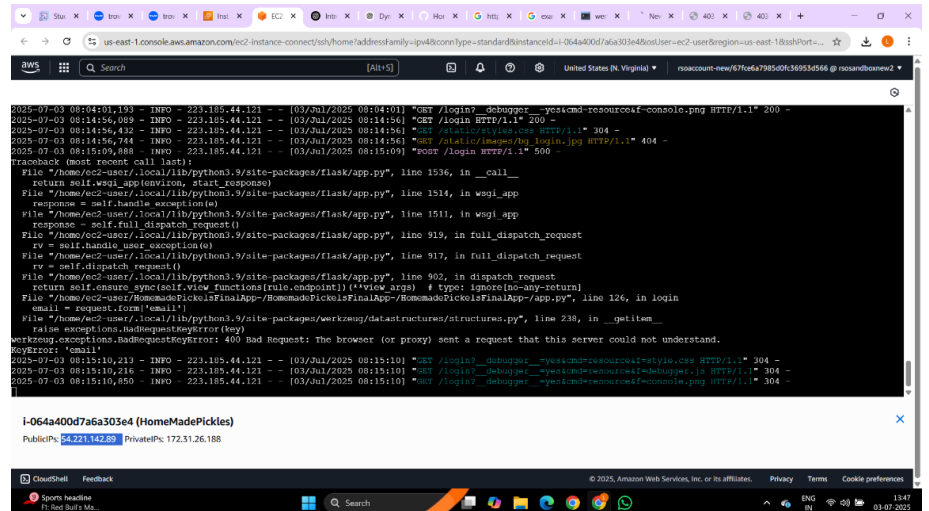
```
sudo yum install python3 git
```

```
sudo pip3 install flask boto3
```

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

```
sudo flask run --host=0.0.0.0 --port=5000
```



```

2025-07-03 08:04:01,193 - INFO - 223.185.44.121 - - (03/Jul/2025 08:04:01) "GET /login?_debugger__yes&cmrresource=console.png HTTP/1.1" 200 -
2025-07-03 08:14:56,089 - INFO - 223.185.44.121 - - (03/Jul/2025 08:14:56) "GET /login HTTP/1.1" 200 -
2025-07-03 08:14:56,432 - INFO - 223.185.44.121 - - (03/Jul/2025 08:14:56) "GET /static/styles.css HTTP/1.1" 304 -
2025-07-03 08:14:56,744 - INFO - 223.185.44.121 - - (03/Jul/2025 08:14:56) "GET /static/images/img_login.jpg HTTP/1.1" 404 -
2025-07-03 08:15:09,888 - INFO - 223.185.44.121 - - (03/Jul/2025 08:15:09) "POST /login HTTP/1.1" 500 -

Traceback (most recent call last):
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
  File "/home/ec2-user/.local/lib/python3.9/site-packages/flask/app.py", line 126, in login
    email = request.form['email']
  File "/home/ec2-user/.local/lib/python3.9/site-packages/werkzeug/datastructures/keyvalues.py", line 238, in __getitem__
    raise exceptions.BadRequestKeyError(key)
werkzeug.exceptions.BadRequestKeyError: 400 Bad Request: The browser (or proxy) sent a request that this server could not understand.
KeyError: 'email'

2025-07-03 08:15:10,213 - INFO - 223.185.44.121 - - (03/Jul/2025 08:15:10) "GET /login?_debugger__yes&cmrresource=style.css HTTP/1.1" 304 -
2025-07-03 08:15:10,216 - INFO - 223.185.44.121 - - (03/Jul/2025 08:15:10) "GET /login?_debugger__yes&cmrresource=debugger.js HTTP/1.1" 304 -
2025-07-03 08:15:10,450 - INFO - 223.185.44.121 - - (03/Jul/2025 08:15:10) "GET /login?_debugger__yes&cmrresource=console.png HTTP/1.1" 200 -
  
```

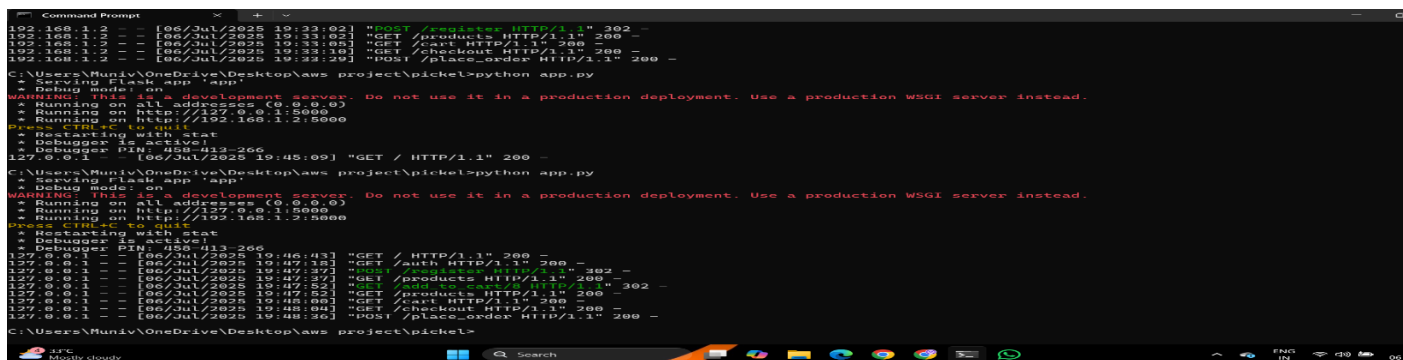
i-064a400d7a6a303e4 (HomeMadePickles)

Public IPs: 54.221.142.89 Private IPs: 172.31.26.188

Verify the Flask app is running:

<http://your-ec2-public-ip>

- Run the Flask app on the EC2 instance



```

192.168.1.2 - - [06/Jul/2025 19:33:02] "GET /register HTTP/1.1" 202 -
192.168.1.2 - - [06/Jul/2025 19:33:02] "GET /products HTTP/1.1" 200 -
192.168.1.2 - - [06/Jul/2025 19:33:03] "GET /cart HTTP/1.1" 200 -
192.168.1.2 - - [06/Jul/2025 19:33:10] "GET /checkout HTTP/1.1" 200 -
192.168.1.2 - - [06/Jul/2025 19:33:20] "POST /place_order HTTP/1.1" 200 -

C:\Users\Muniv\OneDrive\Desktop\aws project\pickle>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://192.0.0.1:5000
* Press CTRL-C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 433-266
127.0.0.1 - - [06/Jul/2025 19:45:09] "GET / HTTP/1.1" 200 -

C:\Users\Muniv\OneDrive\Desktop\aws project\pickle>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://192.0.0.1:5000
* Press CTRL-C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 433-266
127.0.0.1 - - [06/Jul/2025 19:46:43] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [06/Jul/2025 19:47:13] "GET /static HTTP/1.1" 200 -
127.0.0.1 - - [06/Jul/2025 19:47:17] "GET /static HTTP/1.1" 202 -
127.0.0.1 - - [06/Jul/2025 19:47:17] "GET /static HTTP/1.1" 200 -
127.0.0.1 - - [06/Jul/2025 19:47:52] "GET /products HTTP/1.1" 200 -
127.0.0.1 - - [06/Jul/2025 19:47:52] "GET /products HTTP/1.1" 200 -
127.0.0.1 - - [06/Jul/2025 19:48:08] "GET /cart HTTP/1.1" 200 -
127.0.0.1 - - [06/Jul/2025 19:48:08] "GET /cart HTTP/1.1" 200 -
127.0.0.1 - - [06/Jul/2025 19:48:36] "POST /place_order HTTP/1.1" 200 -
127.0.0.1 - - [06/Jul/2025 19:48:36] "POST /place_order HTTP/1.1" 200 -

C:\Users\Muniv\OneDrive\Desktop\aws project\pickle>
  
```

Access the website through:

Public IPs: <http://54.221.142.89/8000>

Milestone 9: Testing and Deployment

- Activity 9.1:

HP HomeMade Pickles & Snacks

Authentic Homemade Flavors

Discover our wide range of traditional pickles and snacks made with love and authentic recipes passed down through generations.

Get Started

44.200.246.95:5000/login

Account Creation Page:

Create Account

Sai1

sai2006@gmail.com

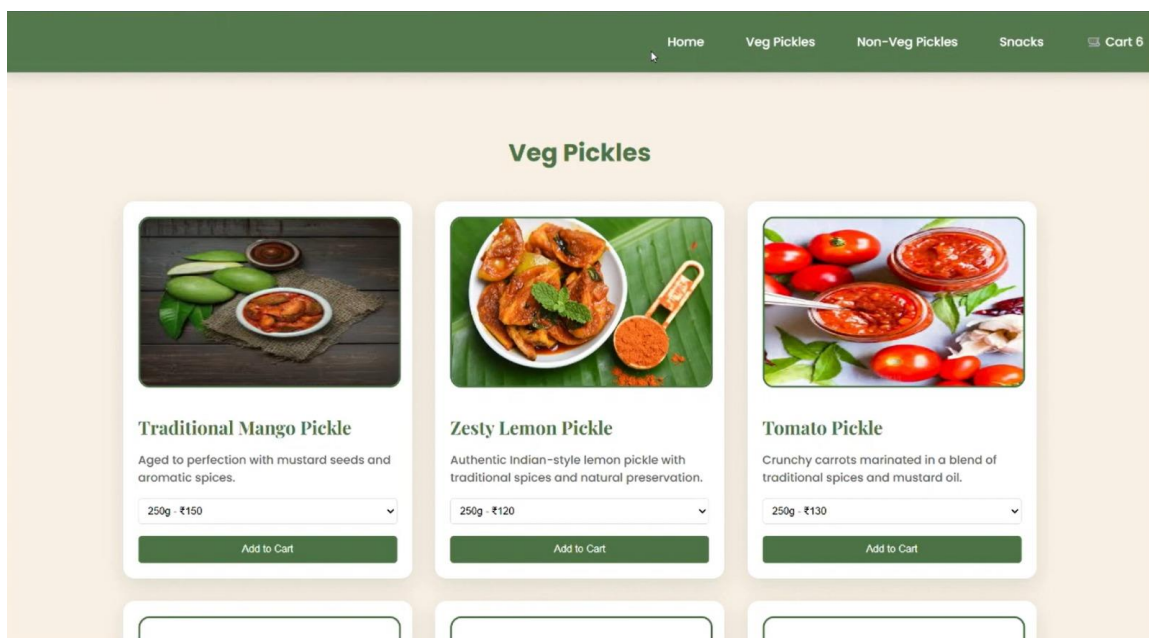
Sign Up

Already have an account? Login

Home page:



Pickels:



Snacks:

Discover our authentic collection of homemade pickles and traditional snacks, each made with love and time-honored recipes.

[All Products](#)
[Pickles](#)
[Snacks](#)


Banana Chips

Fresh

Banana Chips
Crispy banana chips

₹80

★★★★★ (4.9)

ADD TO CART



Murukku

Fresh

Murukku
Traditional murukku

₹100

★★★★★ (4.9)

ADD TO CART



Coconut Chutney Powder

Fresh

Coconut Chutney Powder
Chutney powder

₹90

★★★★★ (4.9)

ADD TO CART

 **Grandma's Pickle Paradise**

Authentic homemade pickles and snacks, made with love and traditional recipes passed down through generations.

Contact Us

+91 9876543210

info@picklesparadise.com

Traditional Kitchen, India

Cart page:

Home Cart					
Item	Price	Quantity		Total	Action
 Mutton Pickle (250g)	₹400	-	1	+	₹400 Remove
 Zesty Lemon Pickle (250g)	₹120	-	2	+	₹240 Remove
 Traditional Mango Pickle (250g)	₹150	-	2	+	₹300 Remove
 Tomato Pickle (1000g)	₹450	-	2	+	₹900 Remove
 Kakarakaya Pickle (500g)	₹240	-	1	+	₹240 Remove
Total Amount: ₹2080					
Proceed to Checkout					

Proceed to Checkout:

[Home](#) [Cart](#)

Total Items: 8
Total Amount: ₹2080

Full Name

Attaluri Naga Venkata Sai Gokul

Delivery Address

25/22-1, Buttai Peta, Machilipatnam, Krishna , Andhra Pradesh, 521001, India

Phone Number

7780145977

Payment Method

Cash on Delivery

Confirm Order

Success:

[Home](#)

 **Order Confirmed Successfully!**

Thank you for choosing HomeMade Pickles!
Your order is being prepared with care and will reach you soon.

Continue Shopping

Exit:

Session Ended

Please close this tab.

Conclusion:

Final Thoughts:

The Homemade Pickles and Snacks platform has been meticulously crafted to deliver a seamless and delightful experience for food enthusiasts seeking authentic, handcrafted flavors. By leveraging modern web technologies such as Flask for backend logic, secure user authentication, and dynamic cart management, the platform ensures a user-friendly interface for browsing, customizing, and ordering artisanal pickles and snacks.

The integration of cloud-ready architecture (e.g., AWS for future scalability) and robust session management allows the platform to handle high traffic efficiently while maintaining real-time updates for orders and inventory. Features like weight-based pricing, category-specific searches, and instant checkout streamline the shopping process, empowering customers to explore a diverse range of traditional and innovative recipes with ease.

This project addresses the growing demand for homemade, preservative-free food products by bridging the gap between small-scale producers and discerning customers. The platform's intuitive design and secure payment workflows enhance trust and convenience, while backend tools enable effortless inventory tracking and order fulfillment for administrators.

By combining time-honored recipes with modern e-commerce capabilities, this website not only preserves culinary heritage but also adapts to the digital age, ensuring that every jar of pickle or snack reaches customers with the same care and quality as a homemade meal. As the platform evolves, it stands ready to scale, introduce new product lines, and foster a community of food lovers united by a passion for authentic flavors.

In essence, this project redefines the way homemade delicacies are shared and enjoyed, offering a flavorful bridge between tradition and technology.